

01 TELEMETRY RECEIVER Specification

- [SMC Telemetry Receiver — Test Plan](#)
 - [1. Objectives](#)
 - [2. Environment](#)
 - [2.1 Multiple DUT geometries](#)
 - [3. Test-bench architecture](#)
 - [4. Primary test list \(\[telemetry_receiver_tb\]\(#\) \)](#)
 - [5. Negative test list \(\[telemetry_receiver_neg_tb\]\(#\) \)](#)
 - [6. Running](#)
 - [7. Coverage](#)
 - [8. AddressSanitizer / UBSan](#)
 - [9. Pass / fail criteria](#)
 - [10. Not covered by these benches](#)

SMC Telemetry Receiver — Test Plan

Verification strategy for the SystemC/TLM-2.0 LT model in

`smc/peripherals/telemetry_receiver`. Benches: `test/telemetry_receiver_tb.cpp` (primary) and `test/telemetry_receiver_neg_tb.cpp` (negative / edge). Both self-check and print `ALL TESTS PASSED`.

1. Objectives

- Confirm the register map (offsets, access types, masks, reset values) matches `telemetry_receiver.rdl`.
- Confirm the ATB decode: beat ordering, packet packing, probe-ID placement, MSB-first counter bytes, per-counter validity, and messages spanning multiple packets — **validated against a hand-derived beat stream**, not against the model's own encoder.
- Confirm message-buffer semantics: FIFO order, `BUFFER_POP`, `STATUS` empty/full, and the RTL's **drop-oldest** overflow policy.
- Confirm both interrupt sources, including their different flavours (level mirror vs. sticky W1C), enable gating at event time, `INTR_TEST` forcing, and threshold truncation.
- Confirm flush behaviour (RX flush of queue + partial message, TX flush handshake in both `afready_i` orders) and reset behaviour.
- Confirm robust bus behaviour: TLM response codes for malformed accesses.
- Achieve ≥ 95 % line coverage of `src/telemetry_receiver.cpp` (achieved: **99 %**, see §7).

2. Environment

- Accellera SystemC 2.3.x / 3.0.x + TLM-2.0, SystemC CCI 1.0, C++17/20.
- A tiny TLM initiator (`driver`) with a `tlm_quantumkeeper` issues 32-bit reads/writes and `transport_dbg` peeks. In the negative bench a `raw_driver` returns the `tlm_response_status` instead of failing the test, so error paths can be asserted on.
- The DUT outputs (`irq_o`, `afvalid_o`, `atready_o`, `debug_o`) are wired to `sc_signal`s observed by the bench; `rst_n_i` and `afready_i` are bench-driven.
- ATB traffic is injected through the `push_atb_beat()` / `push_atb_beats()` back door; streams are built with `telemetry_encode_message()` except where an independent oracle is required (§4, test 3).

2.1 Multiple DUT geometries

Because the assembly buffer and message queue are sized by **immutable** CCI parameters, geometry coverage needs several DUT instances rather than run-time reconfiguration. Four are instantiated across the two benches:

Bench	Instance	buffer_depth	max_counters_per_message	Packets / message	Purpose
primary	tb.tel	4	4 (default)	3	Multi-packet messages, queue behaviour, interrupts
primary	tb.tel1	2	1	1	Single-packet message → hand-derivable beat oracle
negative	tb.telmin	2 (minimum)	1	1	Error paths, empty-pop, threshold truncation
negative	tb.telmax	2	32 (maximum)	19	Widest message; counter array boundary

3. Test-bench architecture

Bench	Focus
telemetry_receiver_tb	Functional happy path: reset, register RW/RO/masking, ATB decode, queue, interrupts, flushes, reset recovery, transport_dbg , CCI.
telemetry_receiver_neg_tb	Error / edge branches: TLM response codes, decode miss, transport_dbg edges, RO write-ignore, empty pop, flush precedence, threshold truncation, extreme geometries.

4. Primary test list (`telemetry_receiver_tb`)

#	Test	Checks
1	Reset values	<code>CTRL=0</code> , <code>STATUS=BUFFER_EMPTY</code> , <code>INTR_*=0</code> , <code>PROBE_ID=0</code> , <code>COUNTER_VLDS=0</code> , <code>COUNTER[0]=0</code> ; <code>irq_o</code> low, <code>afvalid_o</code> low, <code>aready_o</code> high, <code>debug_o=BUFFER_EMPTY</code> , <code>fill_level()</code> =0.
2	Register RW / RO / masking	<code>CTRL.BUFFER_THRESHOLD</code> stores all 12 bits; the <code>BUFFER_POP</code> / <code>RX_FLUSH</code> pulse bits read back 0; <code>INTR_ENABLE</code> masks reserved bits to 0x11; <code>INTR_TEST</code> keeps only the level bit; writes to <code>STATUS</code> , <code>PROBE_ID</code> , <code>COUNTER_VLDS</code> , <code>COUNTER[i]</code> are ignored.
3	Hand-derived beat stream	A beat stream derived by hand from <code>telemetry_receiver_pkg.sv</code> (probe 0x15, counter[0]=0xDEADBEEF, packet word 0xD53BDADDF7BC0000) decodes to the expected probe ID, VLDS=0x1, and counter value — and <code>telemetry_encode_message()</code> reproduces that same stream byte-for-byte. This is the one test that pins the wire format independently of the model.
4	Multi-packet message	4 counters = 3 packets = 24 beats; all four counter values, VLDS=0xF, probe ID; <code>COUNTER[4]</code> and <code>COUNTER[31]</code> read 0 (tied off above <code>max_counters_per_message</code>).
5	Per-counter valid bits	Counters 1 and 3 encoded invalid → VLDS=0x5 and those registers read 0, while 0 and 2 carry their values; full 5-bit probe ID (0x1F).
6	FIFO order / <code>BUFFER_POP</code> / <code>STATUS</code>	4 messages fill the queue → <code>STATUS=BUFFER_FULL</code> , <code>debug_o=BUFFER_FULL</code> ; popping yields them oldest-first ; after draining, <code>STATUS=BUFFER_EMPTY</code> and the message view reads 0.
7	Drop-oldest overflow	6 messages into a 4-deep queue → <code>fill_level()</code> =4 and the visible message is the 3rd (first two dropped), proving the newest telemetry survives.
8	<code>BUFFER_THRESHOLD</code> interrupt	Threshold 1: no IRQ at fill 1, IRQ at fill 2; <code>INTR_STATUS</code> mirrors it; a W1C write does not clear a level source; a bare <code>CTRL=BUFFER_POP</code> write clobbers the threshold to 0 and keeps the IRQ asserted, while a threshold-preserving pop clears it; clearing <code>INTR_ENABLE</code> masks the line while still over threshold.
9	<code>INTR_TEST.BUFFER_THRESHOLD</code>	Forces the IRQ with an empty queue; writing 0 releases it.
10	Missing-last event + recovery	A message with no <code>last_packet</code> is discarded (<code>fill_level()</code> =0), raises the IRQ, sets <code>INTR_STATUS.MISSING_LAST</code> and <code>debug_o</code> bits 0 and 3; W1C clears status and IRQ; the next well-formed message then decodes normally.
11	Missing-last enable gating	With <code>INTR_ENABLE=0</code> the event sets no status and no IRQ, but <code>debug_o[0]</code> still records it.

#	Test	Checks
12	<code>INTR_TEST.MISSING_LAST</code>	Ignored while the enable is clear; with the enable set it latches the sticky status and raises the IRQ; the test bit itself never reads back (pulse); W1C clears.
13	RX flush	Flushes 2 queued messages and a one-packet partial assembly; <code>STATUS=BUFFER_EMPTY</code> afterwards and the next message decodes cleanly (proving the stale beats are gone, not merely hidden).
14	TX flush handshake	With <code>afready_i</code> low: <code>afvalid_o</code> asserts and <code>CTRL.TX_FLUSH</code> reads 1; asserting <code>afready_i</code> retires the request and self-clears the bit. With <code>afready_i</code> already high, the flush retires in the same write.
15	Reset clears state / refuses beats	Reset with a queued message and <code>INTR_ENABLE</code> set: <code>atready_o</code> de-asserts, <code>push_atb_beat()</code> and <code>push_atb_beats()</code> are refused, and after release the queue and registers are back to reset values.
16	<code>transport_dbg / dbg_reg</code> peek	Debug reads return 4 and the right values without disturbing the queue; <code>dbg_reg()</code> on an unmapped offset returns 0.
17	CCI introspection	<code>buffer_depth / max_counters_per_message</code> presets visible per instance and flagged <code>is_preset_value()</code> ; <code>access_delay_ns</code> preset readable and mutable through a broker handle; full parameter list dumped.

5. Negative test list (`telemetry_receiver_neg_tb`)

#	Test	Expected
1	Ignore command	<code>TLM_COMMAND_ERROR_RESPONSE</code>
2	Wrong width (8, 2, 1 bytes)	<code>TLM_BURST_ERROR_RESPONSE</code>
3	Out-of-window (<code>0x100</code> , <code>0x1000</code>) and misaligned (<code>0x02</code> , <code>0x0A</code>)	<code>TLM_ADDRESS_ERROR_RESPONSE</code>
4	In-window decode miss (<code>0x1C</code> , <code>0x20</code> , <code>0x40</code> , <code>0x7C</code> ; read + write)	<code>TLM_ADDRESS_ERROR_RESPONSE</code> , while <code>COUNTER[0]</code> and <code>COUNTER[31]</code> return <code>TLM_OK_RESPONSE</code>
5	<code>transport_dbg</code> rejects	bad length / misaligned / out-of-window / ignore command / write to unmapped offset → returns 0
6	<code>transport_dbg</code> accepts	write + read back of <code>INTR_ENABLE</code> → returns 4, value updates; write to RO <code>COUNTER[1]</code> and <code>STATUS</code> returns 4 but leaves the value at 0
7	<code>dbg_reg</code> guards	<code>dbg_reg(0x100)</code> (past the counter array) and <code>dbg_reg(0x1C)</code> return 0 instead of indexing out of range
8	<code>BUFFER_POP</code> on an empty queue	Ignored three times over — no pointer underflow; <code>STATUS</code> stays <code>BUFFER_EMPTY</code> and the queue still works afterwards
9	Flush precedence	A single <code>CTRL</code> write setting both <code>RX_FLUSH</code> and <code>BUFFER_POP</code> discards the whole queue, not one entry
10	Threshold truncation	With <code>buffer_depth=2</code> (compare width 2 bits) a programmed threshold of <code>0x5</code> behaves as 1 : no IRQ at fill 1, IRQ at fill 2
11	Maximum geometry (32 counters)	<code>telemetry_packets_per_message(32) == 19</code> ; a 152-beat message is fully accepted; all 32 counter values decode, <code>COUNTER_VLDS = 0xFFFFFFFF</code>
12	Missing-last on the maximum geometry	A 19-packet message with no marker raises the sticky IRQ; <code>W1C</code> clears it

6. Running

```
./run_tests.sh           # Release build + run both benches
./run_tests.sh --ctest   # via ctest
./run_tests.sh --asan     # AddressSanitizer + LeakSanitizer (separate build)
./run_tests.sh --coverage # coverage + line report           (separate build)
```

`--asan` and `--coverage` are mutually exclusive and each use an isolated build directory (`build_asan/` , `build_cov/`), per `.cursor/rules/test-coverage-asan.mdc` .

7. Coverage

`./run_tests.sh --coverage` reports:

File	Lines	Covered	%
include/telemetry_receiver.h	21	21	100 %
src/telemetry_receiver.cpp	355	352	99 %

The three uncovered lines in `src/telemetry_receiver.cpp` are:

- the two `SC_REPORT_FATAL` bodies for the constructor's CCI range checks (`buffer_depth >= 2`, `max_counters_per_message` in 1..32). Reaching them aborts elaboration, which cannot be done in-process without leaving the SystemC module hierarchy in an inconsistent state — these are exactly the “defensive guards behind `SC_REPORT_FATAL` invariants” the coverage rule exempts;
- the closing brace of `decode_message()`, where gcov attributes the exception-unwind path of returning a `telemetry_message` by value (an artefact, not a reachable branch).

Both benches contribute: the primary bench covers the functional paths, the negative bench covers the four TLM error branches, the in-window decode miss, the `transport_dbg` rejects, the empty-pop guard, and the two extreme geometries.

8. AddressSanitizer / UBSan

`./run_tests.sh --asan` produces **no ASan or UBSan errors and no leaks attributable to the model** — no frame in any report points into `src/telemetry_receiver.cpp`.

Two environment notes for anyone reproducing this locally:

1. **RHEL 8 / gcc-toolset cannot link ASan** (`cannot find -lasan`): the ASan runtime is not packaged for gcc-toolset-12/13, which is also why `.github/workflows/ci-rhel8.yml` skips the ASAN phase. Use the Ubuntu CI job, or a local `clang++` (`CXX=/usr/bin/clang++ ./run_tests.sh --asan`).
2. **A QuickThreads SystemC build is not ASan-compatible**. SystemC's default coroutine package switches stacks in assembly, which ASan cannot follow: the first `sc_signal::write()` inside any `SC_THREAD` faults with a spurious `SEGV` in `qt_abort`. This is not model-specific — the `beu` bench fails identically. Build SystemC with `-DENABLE_PTHREADS=ON` (or `--enable-pthreads`) for local ASan runs.

The only leak reports that remain come from the global CCI broker that every peripheral bench in this repo creates and intentionally never deletes (`cci_register_broker(new cci_utils::consuming_broker(...))`) plus allocations inside `libcci-config.so` itself. `beu` reports the same pattern from the same line of its own `sc_main`.

9. Pass / fail criteria

- Each bench increments a failure counter on any failed `EXPECT_*` and prints `ALL TESTS PASSED` iff the counter is zero (exit code 0).
- CTest matches `ALL TESTS PASSED / FAILURE(S)` via the regexes in `test/CMakeLists.txt`.
- CI regression is driven by `smc/run_all_smc_tests.sh telemetry_receiver` (Release, ASAN, Coverage, CTest stages) and by the `smc-unit-tests` matrix in `.github/workflows/ci.yml` / `ci-rhel8.yml`.

10. Not covered by these benches

- **Platform integration.** Wiring the receiver into `smc-vp` (address decode at the yet-to-be-agreed wrapper base, PLIC interrupt line, three instances) and a firmware-level drain test are out of scope here; they follow the pattern of `beu/doc/04_BEU_Platform_Integration_Test_Plan.md` and depend on resolving the address-map conflict in `01_TELEMETRY_RECEIVER_Specification.md` §2.1.
- **ATB protocol timing / back-pressure**, since there is no ATB port (`atready_o` reflects reset only).
- **Transmitter behaviour.** The TX flush handshake is checked from the receiver's side only; no transmitter model exists.